ResNets are robust to the deletion of layers even when trained without stochastic depth, while CNN architectures without skip connections are not robust in this way. These results strongly support the idea that adjacent layers in a ResNet encode fundamentally similar representations (Greff et al., 2016).

To the best of our knowledge, this experiment had never been replicated before in transformers. We did so and report our results in Figure 19 above. We find that only the very first layer is crucial for performance; every other layer ablation induces a nearly imperceptible increase in perplexity. Interestingly, Veit et al. (2016) also found that the first layer is exceptionally important in ResNets, suggesting that this is a general phenomenon.

D. Causal intervention details

D.1. Ablation techniques

Prior work has assumed that, given a representation $\boldsymbol{x}$ and a linear subspace $B \subseteq \mathbb{R}^D$ along which we want to remove information, the best erasure strategy is to project $\boldsymbol{x}$ onto the orthogonal complement of B:

$$\boldsymbol{x}' = \mathsf{proj}_{B_\perp}(\boldsymbol{x}) \tag{20}$$

This "zeroes out" a part of $\boldsymbol{x}$, ensuring that $\langle \boldsymbol{x}', \boldsymbol{u} \rangle = \mathbf{0}$ for all $\boldsymbol{u}$ in B. But recent work has pointed out that setting activations to zero may inadvertently push a network out of distribution, making the experimental results hard to interpret (Wang et al., 2022; Chan et al., 2022).

For example, a probe may rely on the invariant that $\forall \boldsymbol{x}, \langle \boldsymbol{x}, \boldsymbol{u} \rangle \gg \mathbf{0}$ for some $\boldsymbol{u}$ in B as an implicit bias term, even if $\langle \boldsymbol{x}, \boldsymbol{u} \rangle$ has low variance and does not encode any useful information about the input. The naive projection $\boldsymbol{x}' = \mathsf{proj}_{B_\perp}(\boldsymbol{x})$ would significantly increase the loss, making it seem that $\mathsf{proj}_B \boldsymbol{x}$ contains important information– when in reality model performance would not be significantly degraded if $\mathsf{proj}_B \boldsymbol{x}$ were replaced with a nonzero constant value more representative of the training distribution.

To correct for this, in our causality experiments we ablate directions by replacing them with their *mean* values computed across a dataset, instead of zeroing them out. Specifically, to ablate a direction $\boldsymbol{u}$, we use the formula:

$$\boldsymbol{x}' = \boldsymbol{x} + P_{\boldsymbol{u}}(\overline{\boldsymbol{x}} - \boldsymbol{x}) \tag{21}$$

where P_u is the projection matrix for $\boldsymbol{u}$ and $\overline{\boldsymbol{x}}$ is the mean representation.

E. Static interpretability analysis

Several interpretability methods aim to analyze parameters that "write" (in the sense of Elhage et al. (2021)) to intermediate hidden states of a language model (Millidge and Black, 2022; Dar et al., 2022; Geva et al., 2022; 2020), and even use this analysis to successfully edit model behavior (Geva et al., 2022; Millidge and Black, 2022; Dai et al., 2022). Given that the tuned lens aims to provide a "less biased" view of intermediate hidden states, we should expect the tuned lens to preserve their effectiveness. We test both static analysis and model editing methods, and confirm this hypothesis. With static analysis, the tuned lens appears to never decrease performance, and for some models, even increases their performance. With model editing, we found the tuned lens to outperform the logit lens on OPT-125m and perform equivalently on other models for the task of toxicity reduction.

E.1. Static Analysis

Many parameters in transformer language models have at least one dimension equal to that of their hidden state, allowing us to multiply by the unembedding matrix to project them into token space. Surprisingly, the resulting top k tokens are often *meaningful*, representing interpretable semantic and syntactic clusters, such as "prepositions" or "countries" (Millidge and Black, 2022; Dar et al., 2022; Geva et al., 2022). This has successfully been applied to the columns of the MLP output matrices (Geva et al., 2022), the singular vectors of the MLP input and output matrices (Millidge and Black, 2022), and the singular vectors of the attention output-value matrices W_{OV} (Millidge and Black, 2022).

In short[7], we can explain this as: many model parameters directly modify the model's hidden state (Elhage et al., 2021), and

[7]The exact interpretation depends on the method used - for instance, the rows of W_{out} can be interpreted as the values in a key-value memory store (Geva et al., 2020).

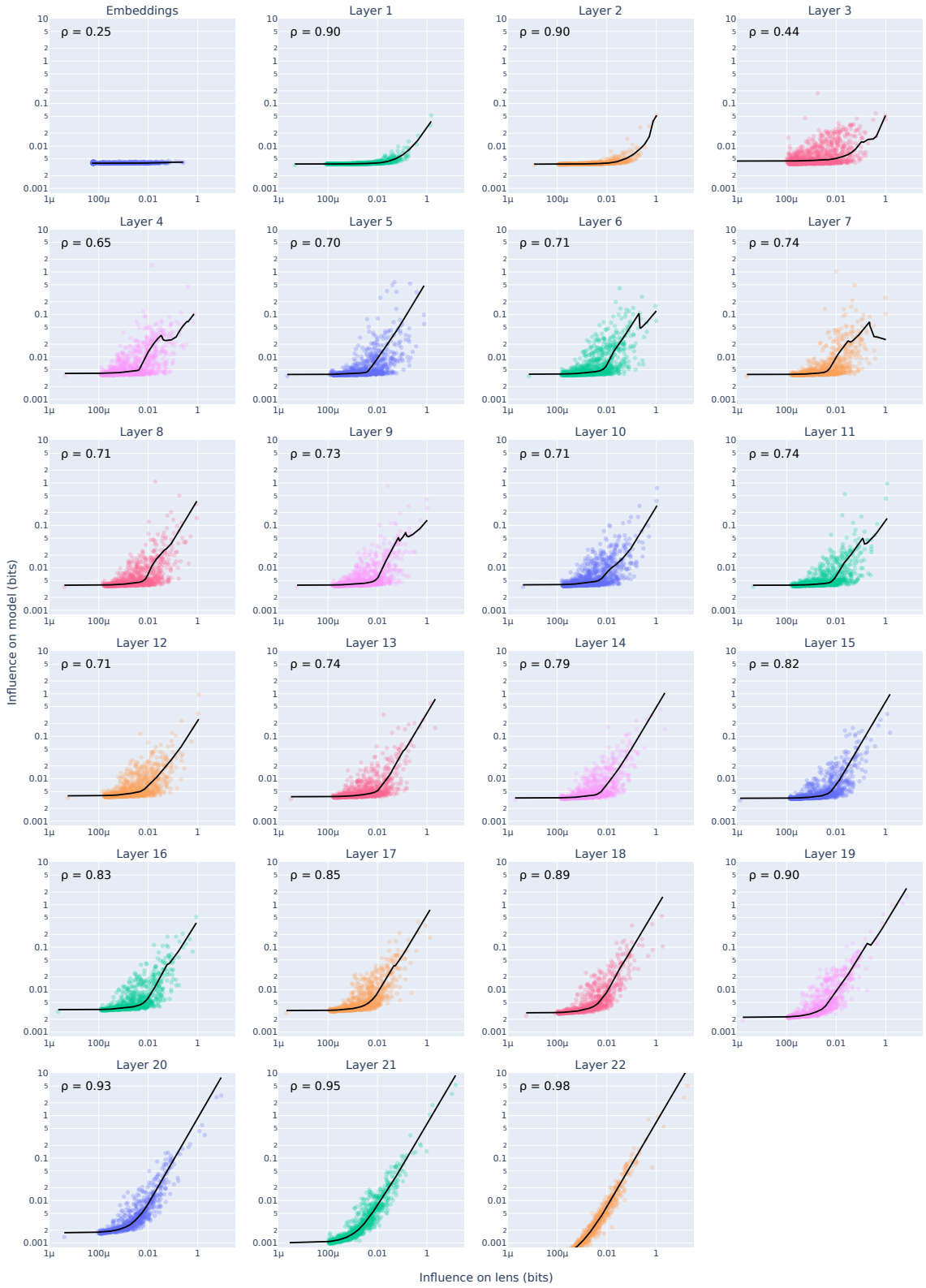

Figure 20. Causal fidelity in Pythia 410M across layers. In the top left corner of each plot is the Spearman rank correlation between the causal influence of each feature on the tuned lens and its influence on the model.

	OV SVD (L)	OV SVD (R)	QK SVD (L)	QK SVD (R)	W_{in} SVD	W_{in} columns	W_{out} SVD	W_{out} rows
Logit lens (real)	0.0745	0.1164	0.0745	0.1164	0.0745	0.0864	0.0728	0.0796
Logit lens (random)	0.0698	0.0695	0.0697	0.0688	0.0689	0.0689	0.0691	0.0688
Tuned lens (real)	0.1240	0.1667	0.1240	0.1667	0.1193	0.1564	0.1196	0.1630
Tuned lens (random)	0.1164	0.1157	0.1177	0.1165	0.1163	0.1262	0.1163	0.1532

Table 3. The mean interpretability scores (as measured in Section E.3) for Pythia 125M, with several different interpretability techniques (Millidge and Black, 2022; Geva et al., 2020), comparing both the tuned lens and logit lens to randomly generated matrices. Where applicable, the notation (L) and (R) indicates that the results are for the left and right singular vectors, respectively.

when viewed from the right angle, the modifications they make are often interpretable. These "interpretable vectors" occur significantly more often than would be expected by random chance (Geva et al., 2022), and can be used to edit the model's behaviour, like reducing probability of specific tokens (Millidge and Black, 2022), decreasing toxicity (Geva et al., 2022), or editing factual knowledge (Dai et al., 2022).

Although these interpretability methods differ in precise details, we can give a generic model for the interpretation of a model parameter W using the unembedding matrix:[8]

$$T(W) = \text{top-}k(f_i(W)W_U)$$

where f_i is some function from our parameter to a vector with the same dimensions as the hidden state, and W_U is the unembedding matrix. In words: we extract a hidden state vector from our parameter W according to some procedure, project into token space, and take the top k matching tokens. The resulting $T(W)$ will be a list of tokens of size k, which functions as a human interpretable view of the vector $f_i(W)$, by giving the k tokens most associated with that vector. As an example, the model parameter W could be the MLP output matrix W_{out} for a particular layer, and f_i the function selecting the ith column of the matrix.

With the tuned lens, we modify the above to become:

$$T(W) = \text{top-}k(L_\ell(f_i(W))W_U)$$

where L_ℓ is the tuned lens for layer number ℓ, projecting from the hidden state at layer ℓ to the final hidden state.

To test this hypothesis, we developed a novel automated metric for evaluating the performance of these parameter interpretability methods, based on the pretrained BPEmb encoder (Heinzerling and Strube, 2018), enabling much faster and more objective experimentation than possible with human evaluation. We describe this method in Appendix E.3.

The results for Pythia 125M can be seen in Table 3. The parameters under both the tuned and logit lens consistently appear more interpretable than random. And the tuned lens appears to show benefit: the difference between random/real average scores is consistently higher with the tuned lens than the logit lens. However, preliminary experiments found much less improvement with larger models, where both the tuned and logit lens appeared to perform poorly.

E.2. Interpretability score distributions

The distribution of interpretability scores for a single parameter (the OV circuit) can be seen in Figure 21. (Interpretability distributions of other parameters appear to share these features as well.) The plot displays a major complication for static analysis methods: most singular vectors are not more interpretable than random, except for a minority lying in a long right tail. It also shows the necessity of comparison to randomly generated matrices: naively comparing the interpretability scores for the tuned lens against those for the baseline would imply a far greater improvement than actual, because the interpretability scores under the tuned lens are higher even for randomly generated matrices.

E.3. Automated interpretability analysis method

While humans can easily tell if a list of words represents some coherent category, this can be subjective and time-consuming to evaluate manually. Some previous work has attempted to automate this process, using GPT-3 in Millidge and Black

[8]In most transformer language models, the function here should technically be $T(W) = \text{top-}k(\text{softmax}(\text{LN}(f_i(W)W_U)))$, including both softmax and LayerNorm. However, the softmax is not necessary because it preserves rank order, and the LayerNorm can be omitted for similar reasons (and assuming that either $f_i(W)$ is zero-mean or that W_U has been left-centered).

[8]Random shuffling applied to each matrix (head-wise for attention matrices), to approximate the element-wise marginal distribution.

[8]Similar to above footnote, random shuffling applied to each W_{OV} for each head.

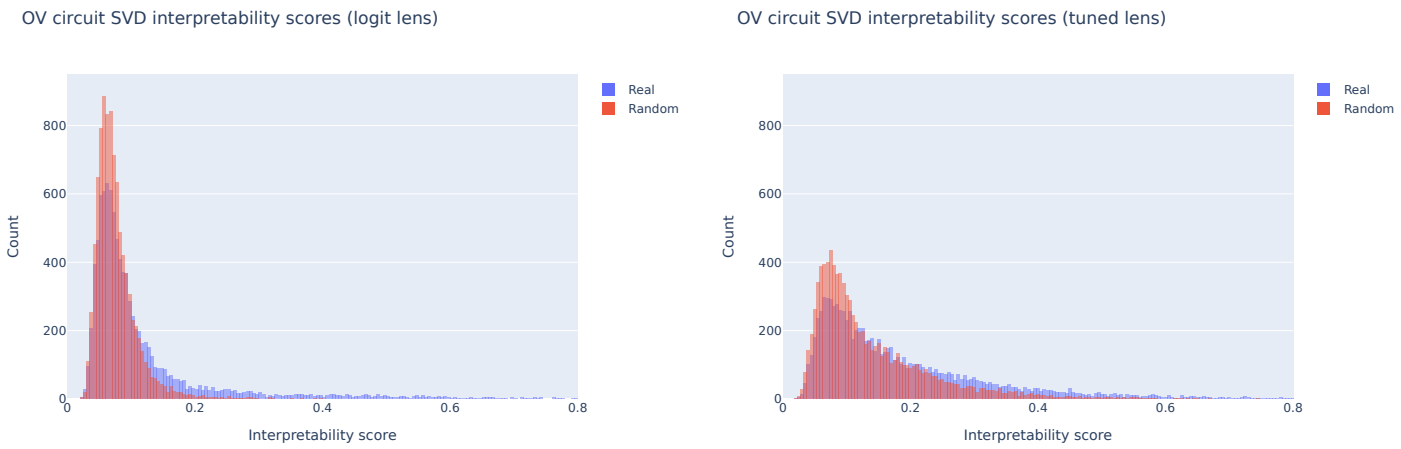

Figure 21. The interpretability scores for the right singular vectors of the OV matrices W_{OV} (following Millidge and Black (2022)) in Pythia 125M, compared with a randomly generated matrix, for both the logit lens (left), and the tuned lens (right).

(2022) or the Perspective API in Geva et al. (2022), but these can still be too slow for quick experimentation.

We created our own method, motivated by the observation that much of the human interpretable structure of these tokens appears to be related to their mutual similarity. Then, the mutual similarity of words can easily be measured by their cosine similarity under a word embedding (Sitikhu et al., 2019). In particular, we use the pretrained BPEmb encoder (Heinzerling and Strube, 2018) [9]. We encode each token, measure the cosine similarity pairwise between all tokens, and take the average value:

$$I(T(W)) = \frac{\sum_{i,j}^{k,k} E(T_i(W)) \cdot E(T_j(W))}{k^2}$$

where the subscript on $T_i(W)$ denotes the i-th token of $T(W)$, and E denotes the normalized BPEmb encoding function. This creates an interpretability metric I that measures something like the "monosemanticity" of a set of tokens.

Because BPEmb is a pretrained model, unlike most CBOW or skip-gram models, there is no ambiguity in the training data used to produce word similarities and results are easy to replicate. We find the interpretability scores given by the model to correspond with human judgment in most cases: it rarely identifies human-uninterpretable lists of tokens as interpretable, although it occasionally struggles to recognize some cases of interpretable structure, like syntactically related tokens, or cases where a list as a whole is more interpretable than a pairwise measure can capture.

E.4. Improved Model Editing

Geva et al. (2022) edit GPT-2-medium to be less toxic by changing the coefficients which feed into the columns of the MLP output matrix, under the key-value framework developed in Geva et al. (2020). They define both the value vector v_i^ℓ (i.e. the columns of the MLP's W_{out}) and a coefficient m_i^ℓ (i.e. the output of W_{in} before the activation function) for each column i and each layer ℓ. This is used to reduce the toxicity of GPT-2-medium by editing the coefficients of specific value vectors. They find:

$$T(W_{\text{out}^\ell}) = \text{top-30}(f_i(W_{\text{out}}^\ell) W_U)$$

where:

$$f_i(W_{\text{out}}^\ell) = v_i^\ell$$

They then concatenate these top-30 tokens for every layer and column, sending it to Perspective API, a Google API service which can return a toxicity score. They then sampled from the vectors that scored < 0.1 toxicity (where a score > 0.5 is classified as toxic), set each value vector's respective coefficient to 3 (because 3 was a higher than average activation), and ran the newly edited model through a subset of REALTOXICPROMPTS (Gehman et al., 2020) to measure the change in toxicity compared to the original model.

[9] We also tried the fastText encoder (Bojanowski et al., 2016), but found BPEmb to give results better matching human judgment.

	Toxicity	Severe Toxicity	Sexually explicit	Threat	Profanity	Identity attack	Perplexity
Original/ Unedited	0.50	0.088	0.159	0.058	0.40	0.043	**27.66**
Logit Lens Top-5	0.47	0.077	0.148	0.057	0.38	0.043	27.68
Logit Lens Top-10	0.45	0.073	0.144	0.056	0.36	0.040	27.70
Logit Lens Top-20	0.43	0.072	0.141	0.057	0.33	0.041	27.73
Tuned Lens Top-5	0.45	0.079	0.143	0.056	0.36	0.041	**27.66**
Tuned Lens Top-10	0.42	0.063	0.143	0.057	0.33	**0.034**	27.67
Tuned Lens Top-20	**0.39**	**0.061**	**0.138**	**0.052**	**0.31**	0.035	27.73

Table 4. Percentage of prompt generations labeled as toxic for OPT-125m. Best results in **bold**. Across all respective settings, the tuned lens outperformed the logit lens with no significant increases in perplexity. Although the value vectors were only graded & ranked by toxicity, there were still decreases in other metrics (severe toxicity, sexually explicit, etc) that were not directly selected against. However, these results did not generalize to other models tested.

Changing the coefficients of value vectors can generate degenerate text such as always outputting " the" which is scored as non-toxic. To measure this effect, they calculate the perplexity of the edited models, where a higher perplexity may imply these degenerate solutions.

We perform a similar experiment, but found the vast majority of value vectors that scored < 0.1 toxicity to be non-toxic as opposed to anti-toxic (e.g. semantic vectors like " the", " and", etc are scored as non-toxic). Alternatively, we set the k most toxic value vector's coefficient to 0, as well as projecting with both the logit and tuned lens on OPT-125m, as seen in Table 4.

We additionally tested pythia-125m, pythia-350m, gpt-2-medium, and gpt-neo-125m; however, there were no significant decreases in toxicity (i.e. $> 2\%$) compared to the original, unedited models.

E.5. Implementation

The initial implementation of many of our static analysis experiments was helped by the use of the `transformer_lens` library (Nanda, 2022).